

Sign-Language Recognition

Training & Inference — Technical Documentation

Generated 2026-06-26

#	Section
01	Overview
02	Architecture
03	Pipeline
04	File Reference
05	Data Formats
06	Developer Guide
07	Results And Future
A	Appendix — Training Metrics

01 — Overview

What this layer does

It turns the **verified landmark dataset** (`processed_dataset/`) into a working **sign-language recogniser**: a small sequence model that reads a clip of skeleton landmarks and outputs one of 12 word classes, plus everything needed to run it on a video file or a live webcam.

The extraction project answered *"are the landmarks correct?"*. This project answers *"given correct landmarks, what sign is being made?"* — without touching the extraction format. Every clip is read as a raw (`T, 1692`) landmark array and run through one shared feature transform, so **what the model trains on is exactly what it sees at serve time** (no train/serve skew).

The five stages

The pipeline is deliberately split into numbered, runnable stages:

Stage	Script	Job	Key output
1	<code>train.py</code>	Stratified k-fold cross-validation — honest whole-dataset accuracy	<code>metrics/cv_report.json</code> , confusion matrix
2	<code>finetune.py</code>	Fit the deployment model on 100% of the data (EMA weights)	<code>checkpoints/model_final.pt</code>
3	<code>export_optimize.py</code>	Export to ONNX + int8, verify parity, measure latency	<code>exported/sign_model.onnx</code>
4	<code>infer_video.py</code>	Run the model on a video file , render a prediction overlay	<code>predictions/*_pred.mp4</code>
5	<code>infer_live.py</code>	Live webcam translation (record -> process -> delete chunks)	<code>predictions/live_transcript.json</code>

Stages 1–3 are offline (training). Stages 4–5 are inference and share one `SignPredictor` so the serving path is identical everywhere.

The model

- **Input:** a fixed-length (`L, D`) feature sequence — `L = 64` frames, `D = 346` features/frame (default config).
- **Network:** a small **2-layer BiGRU** with attention pooling (a Transformer encoder is available via config). ~0.5 M parameters.
- **Output:** 12-way softmax over the word classes.
- **Measured:** ~**0.91** out-of-fold accuracy / macro-F1 (5-fold CV).

Skeleton features are low-dimensional, so the model is intentionally small and heavily regularised (dropout, label smoothing, augmentation, EMA).

The dataset (consumed read-only)

- **12 word classes:** ``good, hello, help, hi, how_are_you, me, nice_to_meet_you, no, sorry, thank_you, yes, you``.

- ~11 clips per class (~130 usable clips after pruning).
- Source: `processed_dataset/landmarks_all/<class>/<stem>.npy`, each (T, 1692), indexed by `manifest.csv` (rows with `verdict == FAIL` are skipped).

With so few clips per class, a single train/val/test split is unreliable — hence k-fold CV for evaluation and a separate full-data fit for deployment.

Tech stack

Layer	Tool	Notes
Language	Python	3.11
Tensors / training	PyTorch	BiGRU / Transformer, AdamW, cosine schedule
Arrays / metrics	NumPy	folds, confusion matrix — no scikit-learn dependency
Deployment runtime	ONNX Runtime	CPU inference (fp32 + dynamic int8)
Detection (inference)	MediaPipe + OpenCV	reused from the extraction project
Charts	matplotlib	confusion / fold / history plots

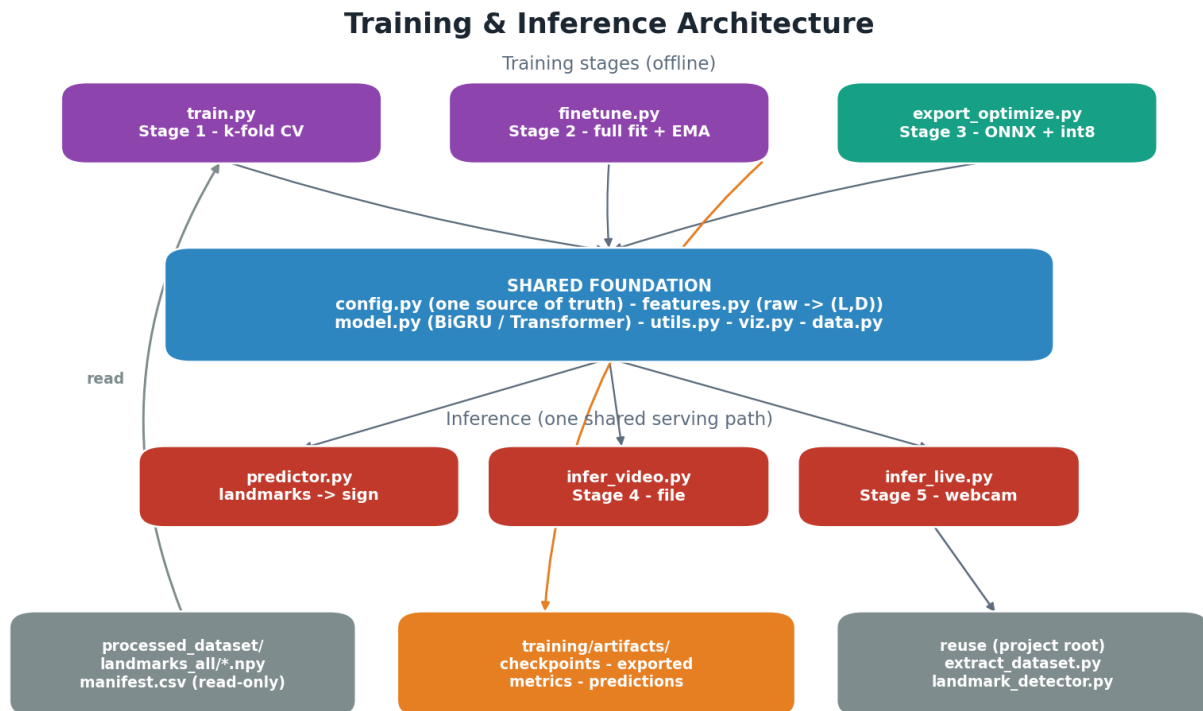
Directory tree

```

training/
+-- config.py          # ONE source of truth: paths, layout, all hyperparameters
+-- features.py        # raw (T,1692) -> fixed (L,D) – shared by train AND inference
+-- data.py            # dataset index, in-memory cache, augmentation, torch Dataset
+-- model.py           # SignClassifier (BiGRU / Transformer) + pooling
+-- utils.py           # seeding, stratified folds, metrics, JSON IO, logging
+-- viz.py             # confusion / fold / history plots
|
+-- train.py           # STAGE 1 – k-fold cross-validation
+-- finetune.py        # STAGE 2 – fit final model on all data (EMA)
+-- export_optimize.py # STAGE 3 – export + optimize to ONNX
+-- predictor.py       # shared inference: load model, landmarks -> sign
+-- infer_video.py     # STAGE 4 – run on a video file + overlay
+-- infer_live.py      # STAGE 5 – live webcam translation
|
+-- artifacts/         # ALL outputs (gitignored, regenerable)
|   +-- checkpoints/   # fold{k}.pt, model_final.pt, model_meta.json
|   +-- metrics/       # cv_report.json + confusion / fold / history PNGs
|   +-- exported/      # sign_model.onnx, sign_model.int8.onnx, model_meta.json
|   +-- predictions/   # *_pred.mp4 / *_pred.json, live_transcript.json
|   +-- cache/         # feature cache
|   +-- live_tmp/      # transient live recordings (auto-deleted)
|   +-- logs/          # train.log, finetune.log
|
+-- docs/              # THIS DOCUMENTATION
+-- *.md

```

02 — Architecture



Layers

The training layer is organised so the **feature transform and the model are each written once** and reused by every stage. Training and inference are not two code paths — they are the same path with augmentation toggled on/off.

1. **Shared foundation** — `config.py`, `features.py`, `model.py`, `utils.py`.

These define *what a feature is*, *what the model is*, and *the conventions*. Everything else imports them.

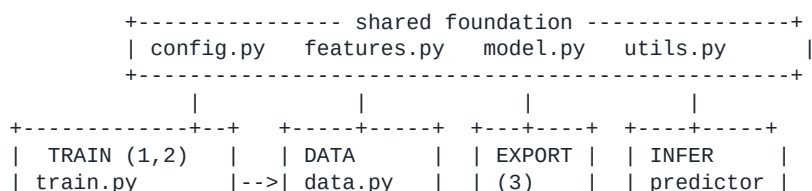
2. **Training stages** — `train.py` (k-fold CV) and `finetune.py` (full-data fit). They wrap the foundation with data loading (`data.py`), augmentation, schedules, EMA, and metrics (`viz.py`).

3. **Deployment** — `export_optimize.py` freezes the trained weights into a framework-free ONNX model with a self-describing `model_meta.json`.

4. **Inference** — `predictor.py` is the single serving entry point; `infer_video.py` and `infer_live.py` are thin CLIs over it.

5. **Cross-project reuse** — inference calls back into the extraction project

(`extract_dataset.extract_clip`, `landmark_detector`) to turn a video into the same (T, 1692) raw vector the dataset was built from.



```

| finetune.py | | viz.py | | ONNX | -> | video(4) |
+-----+ +-----+ +-----+ | live (5) |
      produces
checkpoints/model_final.pt --> exported/*.onnx --> reuses extract_dataset

```

Module responsibilities

Module	Responsibility	Depends on
<code>config.py</code>	Paths, raw landmark layout, all hyperparameters (dataclasses).	stdlib only
<code>features.py</code>	<code>(T, 1692) -> (L, D)</code> : normalise, select, resample, velocity, masks.	NumPy, config
<code>data.py</code>	Index dataset, cache geometry, augment, expose a torch <code>Dataset</code> .	torch, features, config
<code>model.py</code>	<code>SignClassifier</code> (BiGRU / Transformer) + attention/mean/last pooling.	torch, config
<code>utils.py</code>	Seeding, stratified folds, confusion/report metrics, JSON, <code>Tee</code> logger.	NumPy
<code>viz.py</code>	Confusion-matrix / per-fold / history plots.	matplotlib
<code>train.py</code>	Stage 1: k-fold CV, OOF metrics, per-fold checkpoints.	all of the above
<code>finetune.py</code>	Stage 2: full-data fit with EMA -> <code>model_final.pt</code> .	train.py, model, data
<code>export_optimize.py</code>	Stage 3: ONNX export + int8 + parity check + latency.	torch, onnxruntime
<code>predictor.py</code>	Load <code>model_meta.json</code> , run ONNX/torch, landmarks -> sign.	features, onnxruntime
<code>infer_video.py</code>	Stage 4: file inference + prediction overlay video.	predictor, extract_dataset
<code>infer_live.py</code>	Stage 5: chunked record->process->delete live loop.	predictor, OpenCV

Design principles

- **One feature transform, no train/serve skew.** `features.to_features()` is

used by training *and* inference. It carries **no fitted state** — the geometry (center on mid-shoulder, scale by shoulder width) is fully deterministic, and the model's input `LayerNorm` absorbs any residual scaling. What you train on is exactly what you serve.

- **Augment in normalised coordinate space.** Because normalisation cancels

global scale/translation, augmentation must act *between* the geometric stage and assembly — so `features.py` is split into `geometric()` -> `assemble()` and `data.py` augments in the middle. The high-value augment is the **horizontal flip**, which turns a right-handed example into a valid left-handed one.

- **Honest evaluation, separate deployment.** `train.py` reports out-of-fold

accuracy over the *whole* dataset (every clip validated exactly once); `finetune.py` then trains the shipped model on 100% of the data. CV measures the recipe; finetune produces the artifact.

- **Self-describing models.** `model_meta.json` records the exact `FeatureConfig`,

class list, dims, and ONNX filenames. Inference rebuilds the feature pipeline from this file alone — no hidden coupling to training-time globals.

- **Reuse the verified extractor at inference.** A live/file video is converted to

(`T`, 1692) by the project's own `extract_clip` + `build_all_vector`, so live results match offline results with no drift-prone second detector.

- **Regenerable artifacts.** Everything under `training/artifacts/` is gitignored

and reproducible by re-running the stages; nothing there is hand-edited.

Feature & coordinate conventions

- Raw input is the dataset's (`T`, 1692) per-frame vector (pose | face | left |

right), MediaPipe normalised image coordinates.

- The transform **drops the face block by default** (1434 of 1692 dims), keeps a

curated set of 13 upper-body pose joints + both hands, and adds velocity and presence-mask channels — yielding `D = 346` (see `05_data_formats.md`).

- Absent landmarks stay at the **origin** (re-zeroed after normalisation), so a

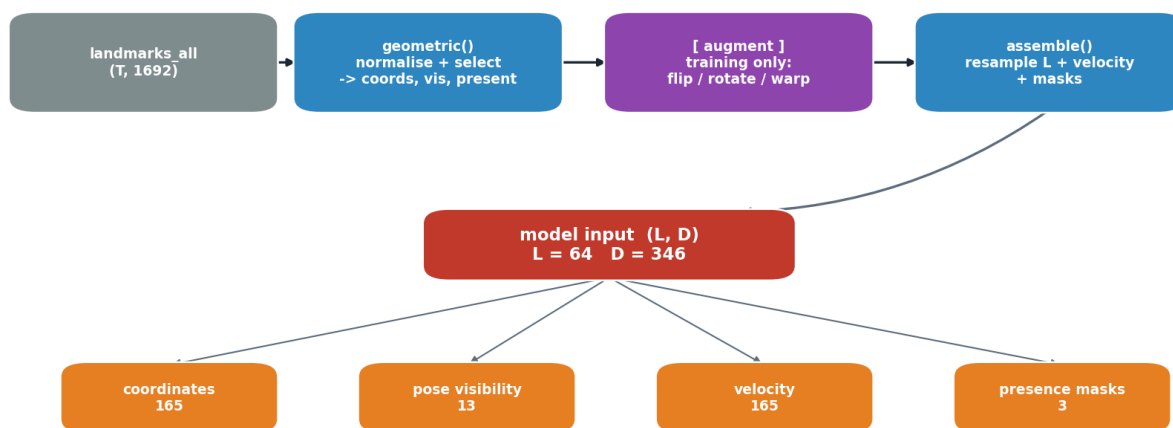
missing hand is never a "ghost" point far from frame.

03 — Pipeline

This is the data flow from a raw landmark clip to a recognised sign: first the shared **feature transform**, then the **five stages**.

```
processed_dataset/landmarks_all/<class>/<stem>.npy    (T, 1692)
|
v features.geometric()      normalise + select -> coords (T,P,3), vis, present
| [data.augment()]          training only: flip / rotate / scale / jitter / time-warp
v features.assemble()       resample to L + velocity + masks -> (L, D)
|
+-----+
| STAGE 1 train.py    k-fold CV      -> cv_report.json      |
| STAGE 2 finetune.py full fit+EMA -> model_final.pt      |
| STAGE 3 export      ONNX + int8    -> sign_model.onnx    |
| STAGE 4 infer_video file + overlay |
| STAGE 5 infer_live  webcam chunks  |
+-----+
```

Feature Transform (raw landmarks -> model input)



55 points x 3 (13 pose joints + 21 left + 21 right) + 13 vis + 165 velocity + 3 masks

Augmentation acts in NORMALISED space (between geometric and assemble) so global scale/translation isn't cancelled away.

The feature transform (features.py)

A clip becomes a fixed (L, D) tensor in three deterministic steps. It is split so augmentation can act in the *normalised coordinate space* (see 02).

Stage 1 — geometric(all_vec) -> coords, vis, present

1. **Slice** the (T, 1692) vector into pose (T, 33, 4), face (T, 478, 3), left hand and right hand (T, 21, 3) (split_blocks).

2. **Presence** per modality per frame: a block is "detected" if its x/y/z is not all-zero (_present).

3. **Spatial normalisation** (_reference + norm_xyz): subtract the **mid-shoulder**

point and divide by the **shoulder width**, using ONE clip-level reference (the median over frames where the pose is present). This gives translation + scale invariance, robust to per-frame dropouts. If shoulders can't be measured, fall back to **fallback_scale = 0.25**.

4. **Select** the enabled points: curated 13 upper-body pose joints + both hands (face off by default). Pose **visibility** is kept as a separate channel.

5. **Re-zero absent blocks** *after* normalisation so a missing hand stays at the origin instead of being pushed to **-origin/scale** (a ghost hand).

Stage 2 — **assemble(coords, vis, present) -> (L, D)**

1. **Temporal resample** every channel to a fixed **L = seq_len** frames — linear

interpolation for coordinates/visibility, nearest-neighbour for masks (keeps 0/1 crisp). Clips are 14–286 frames; all become exactly **L**.

2. **Velocity**: append the first-difference (frame-to-frame motion) of the coordinates.

3. **Masks**: append the per-modality presence channels (pose / left / right).

4. **Concatenate** -> **(L, D)**. For the default config, **D = 346**:

165 (coords) + 13 (vis) + 165 (velocity) + 3 (masks).

to_features() is the no-augment wrapper used at inference; **feature_dim()** computes **D** for a config without needing real data.

Flip permutations (build_spec / FeatureSpec)

build_spec() precomputes the index permutations used by the horizontal-flip augment: swap left/right hand points, swap symmetric pose joints (**POSE_FLIP_PAIRS**), and swap the two hand mask columns. Flipping is an involution (flip twice = identity), checked in **features.py**'s self-test.

Augmentation (data.augment, training split only)

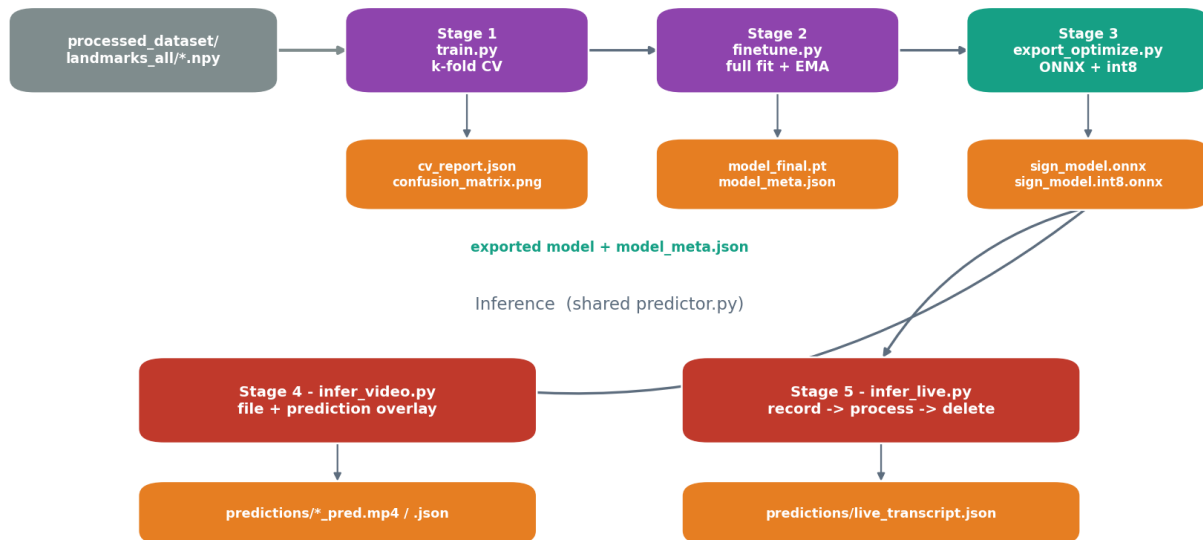
Applied in normalised space, between **geometric** and **assemble**:

- **Temporal**: per-frame dropout, then a random sub-window (speed warp).
- **Horizontal flip** (**aug_flip_prob = 0.5**): point permutation + negate x + swap hand presence — a right-handed clip becomes a valid left-handed one.
- **Geometric**: small rotation (+/-13°), scale (+/-12%), translation (+/-0.06),

Gaussian jitter — then absent points are re-zeroed so they stay at the origin.

data.load_geometric() runs **geometric()** **once per clip** and caches the result in memory; each epoch only pays for augment + assemble.

Pipeline Stages



Stage 1 — train.py (k-fold cross-validation)

1. `stratified_folds()` splits clips into $k = 5$ validation folds, each class spread evenly.
2. For each fold: train on the other folds (augmentation on), validate on the held-out fold (augmentation off), keep the **best val-acc weights**, and record the fold's validation predictions.
3. Stitched together, the per-fold validation predictions are the ****out-of-fold (OOF)**** prediction for every clip — an honest whole-dataset accuracy + confusion matrix.
4. Schedule: AdamW + linear **warmup** then **cosine decay** to `min_lr`; gradient clipping; early stopping on val-acc patience.
5. Writes `cv_report.json`, `confusion_matrix.png`, `cv_folds.png`, `history.png`, and per-fold `fold{k}.pt`.

Stage 2 — finetune.py (deployment fit)

1. Train on **100% of the clips** (no held-out set) with the same augmentation and schedule.
2. Maintain an **Exponential Moving Average (EMA)** of the weights with decay `warmup(min(0.999, (1+step)/(10+step))`) so the average tracks closely on a short, small-data run.
3. At the end, evaluate EMA vs raw weights on the (no-aug) training clips and **ship whichever fits better** (EMA usually wins) — guaranteeing the shipped model is never worse than the last raw step.
4. Optionally **warm-start** from a fold checkpoint (`--init-from fold3`).
5. Writes `model_final.pt` (weights + full self-describing meta) and

`model_meta.json`.

Stage 3 — `export_optimize.py` (ONNX)

1. Load `model_final.pt`, rebuild the network, load weights.
2. **Export to ONNX** (opset 17; batch axis dynamic, sequence length fixed at `L`).
3. **Verify** ONNX Runtime outputs $\approx$ PyTorch outputs (random batches + a real clip); reports max abs logit diff (observed $\sim 1.7e-6$).

4. **Dynamic int8 quantization** -> smaller/faster CPU model; re-verify top-1 class agreement with fp32.

5. Copy `model_meta.json` beside the ONNX (with `onnx` / `onnx_int8` filenames) so inference needs nothing else. Report file sizes + measured CPU latency.

Stage 4 — `infer_video.py` (file inference)

1. `video_to_clip()` runs the project's **own** `extract_clip` + `build_all_vector` -> the same `(T, 1692)` raw vector the dataset uses.
2. `SignPredictor.predict()` -> whole-clip class + top-k.
3. `timeline()` slides a window (default 48 frames, stride 12) for a per-segment "subtitle" sequence; windows below `CONF_THRESH = 0.45` show "...".
4. `render_overlay()` draws the saved landmarks on the original frames (same look as the dataset's verify overlay) with the prediction as a banner subtitle -> `predictions/<stem>_pred.mp4 + .json`.
5. `--scan-dataset` batch-tests the first N source videos per class.

Stage 5 — `infer_live.py` (live translation)

The requested **record** -> **process** -> **delete** loop:

1. Record a few seconds of webcam (`--seconds`, default 2.5) to a **temp file** under `live_tmp/`.
2. Run the **same** `video_to_clip` + predictor on that recording.
3. Show the recognised sign as a subtitle; emit it only if confidence $\geq$ `--conf` **and** hands were actually seen ($>15\%$ of frames).
4. **Delete** the temp file (it is a cache file). Repeat continuously.
5. `--source <video>` simulates the live loop from a file (testable without a camera); a transcript is saved to `predictions/live_transcript.json`.

Using a temp file (not a pure in-memory buffer) is deliberate: it reuses the exact, already-verified extraction path, so live output matches offline output.

04 — File Reference

Every Python file in `training/`, what it contains, and the logic of its key pieces.

`config.py` — single source of truth (~140 lines)

Pure data + path logic; imports nothing heavy, so it is safe to import anywhere (including live inference). Paths resolve relative to the **project root** (the parent of `training/`), never the current working directory.

Symbol	Role
Path constants	<code>PROCESSED</code> , <code>LANDMARKS_ALL</code> , <code>MANIFEST</code> , <code>CLASSES_JSON</code> , and the <code>artifacts/</code> subdirs (<code>CKPT_DIR</code> , <code>METRICS_DIR</code> , <code>EXPORT_DIR</code> , <code>PREDICTIONS_DIR</code> , <code>LIVE_TMP_DIR</code> , <code>LOG_DIR</code> , <code>CACHE_DIR</code>).
<code>RAW_TOTAL</code> , <code>BLOCK</code>	The <code>(T, 1692)</code> raw layout (mirrors <code>processed_dataset/FEATURE_LAYOUT.json</code>).
<code>UPPER_POSE_IDX</code>	The 13 curated signing-relevant pose joints (nose, shoulders, elbows, wrists, hand roots).
<code>POSE_FLIP_PAIRS</code> , <code>LEFT/RIGHT_SHOULDER</code>	L/R symmetric joints (for flip) and the normalisation reference.
<code>FeatureConfig</code>	How a clip becomes <code>(L, D)</code> : <code>seq_len</code> , <code>use_pose/hands/face</code> , <code>add_velocity</code> , <code>add_masks</code> , <code>normalize</code> , etc.
<code>ModelConfig</code>	<code>arch</code> (<code>bigru/transformer</code>), <code>hidden</code> , <code>layers</code> , <code>dropout</code> , <code>pool</code> , transformer dims.
<code>TrainConfig</code>	Optimisation + all augmentation knobs (<code>epochs</code> , <code>lr</code> , <code>folds</code> , <code>aug_*</code> , early stop).
<code>Config</code> , <code>CONFIG</code>	Composite of the three; <code>CONFIG</code> is the shared default instance.
<code>ensure_dirs()</code>	Create all artifact dirs.
<code>to_dict()</code> / <code>feature_config_from_dict()</code>	Round-trip config to/from JSON (rebuilds tuple fields).

`features.py` — shared preprocessing (~294 lines)

Raw `(T, 1692)` -> fixed `(L, D)`. No fitted state. Split into `geometric` -> `assemble` so augmentation can act in normalised space. **Used by training AND inference.**

Symbol	Role
<code>FeatureSpec</code>	Precomputed layout: point segments, flip permutations (points/vis/masks), counts. <code>point_present()</code> expands per-modality presence to per-point.
<code>build_spec(cfg)</code>	Build the <code>FeatureSpec</code> (segments + flip permutations) for a config.
<code>split_blocks(all_vec)</code>	<code>(T, 1692)</code> -> dict of <code>(T, N, C)</code> arrays per modality.
<code>_present(block)</code>	Per-frame "detected" = block not all-zero.
<code>_reference(...)</code>	Clip-level origin (median mid-shoulder) + scale (median shoulder width).
<code>_resample_linear</code> / <code>_resample_nearest</code>	<code>(T, D)</code> -> <code>(L, D)</code> time resampling.
<code>geometric(all_vec, cfg, spec)</code>	Stage 1: normalise + select -> <code>coords</code> , <code>vis</code> , <code>present_modal</code> .

Symbol	Role
<code>assemble(coords, vis, present, cfg, spec)</code>	Stage 2: resample + velocity + masks -> (L, D).
<code>to_features(all_vec, cfg, spec)</code>	Inference wrapper (geometric->assemble, no augment).
<code>feature_dim(cfg, spec)</code>	Compute D for a config (no real data needed).

Self-test (`__main__`) prints the spec, transforms a sample clip, and asserts the flip is an involution.

data.py — dataset + augmentation (~173 lines)

Symbol	Role
<code>load_class_names()</code>	Class names ordered by integer label (from <code>classes.json</code>).
<code>load_index(drop_fail)</code>	One dict per usable clip from <code>manifest.csv</code> (skips <code>FAIL</code>); falls back to a glob if no manifest.
<code>load_geometric(...)</code>	Load every clip, run <code>geometric()</code> once, cache (coords, vis, present) in memory.
<code>augment(coords, vis, present, ...)</code>	Training-only: temporal dropout/warp, horizontal flip (+L/R swap), rotation, scale, translation, jitter; re-zero absent points.
<code>SignDataset</code>	torch <code>Dataset</code> ; indexes the cache, augments if enabled, runs <code>assemble()</code> -> (features, label).

model.py — sequence models (~110 lines)

Input (B, L, D) -> logits (B, C). Small + regularised; all ops ONNX-exportable.

Symbol	Role
<code>AttentionPool</code>	Additive attention pooling over time -> (B, dim).
<code>_Pool</code>	Pooling selector: <code>attention</code> / <code>mean</code> / <code>last</code> .
<code>SignClassifier</code>	input <code>LayerNorm</code> -> MLP -> BiGRU *or* Transformer encoder -> pool -> head.
<code>build_model(in_dim, n_classes, cfg)</code>	Construct a <code>SignClassifier</code> .
<code>count_params(model)</code>	Trainable parameter count.

The input `LayerNorm` is what lets preprocessing stay fitted-state-free — it absorbs any residual feature scaling.

utils.py — shared helpers (~122 lines)

No scikit-learn dependency — metrics are NumPy.

Symbol	Role
<code>_enable_utf8_console()</code>	Make stdout/stderr UTF-8 tolerant on Windows (runs on import).
<code>seed_everything(seed)</code>	Seed <code>random</code> / NumPy / torch.
<code>stratified_folds(labels, k)</code>	k validation-index arrays, each class spread evenly.
<code>confusion_matrix</code> / <code>classification_report</code>	Accuracy + per-class P/R/F1 + confusion matrix.
<code>save_json</code> / <code>load_json</code>	JSON IO (creates parent dirs).

Symbol	Role
<code>Tee</code>	Print to stdout AND append to a log file.

viz.py — plots (~71 lines)

Headless matplotlib (Agg).

Symbol	Role
<code>plot_confusion(cm, names, path)</code>	Row-normalised confusion-matrix heatmap.
<code>plot_cv_folds(fold_accs, path)</code>	Per-fold accuracy bars + mean line.
<code>plot_history(histories, path)</code>	Train-loss + val-accuracy curves per fold.

train.py — STAGE 1: k-fold CV (~252 lines)

Symbol	Role
<code>get_device()</code>	CUDA if available, else CPU.
<code>make_scheduler(...)</code>	Linear warmup -> cosine decay to <code>min_lr</code> .
<code>evaluate(model, loader, ...)</code>	Preds / targets / loss / accuracy (no grad).
<code>train_model(train_ds, val_ds, ...)</code>	Train one model; best-val-acc weights + history; supports a no-val "monitor" mode used by finetune.
<code>run_cv(cfg, args)</code>	Loop folds, gather OOF predictions, write report + plots + per-fold checkpoints.
<code>apply_overrides / parse_args</code>	CLI: <code>--epochs --folds --batch-size --lr --seq-len --arch --hidden --use-face --no-aug --seed</code> .

finetune.py — STAGE 2: deployment fit (~188 lines)

Symbol	Role
<code>EMA</code>	Exponential moving average of weights, with decay warmup for short runs.
<code>build_meta(...)</code>	Self-describing meta dict (classes, dims, config, timestamp).
<code>fit_final(cfg, args)</code>	Train on all data with EMA; ship the better of EMA/raw; write <code>model_final.pt</code> + <code>model_meta.json</code> (and fold in CV accuracy if present).
<code>parse_args</code>	Same overrides as train + <code>--init-from <fold></code> warm-start.

export_optimize.py — STAGE 3: ONNX export (~144 lines)

Symbol	Role
<code>load_final(device)</code>	Load <code>model_final.pt</code> , rebuild network + feature config.
<code>_real_clip_features(...)</code>	A real clip's features for the parity check.
<code>export(args)</code>	ONNX export (opset 17, dynamic batch) -> verify vs PyTorch -> int8 quantize -> verify agreement -> write meta -> report sizes + latency.

Symbol	Role
<code>parse_args</code>	<code>--no-quant</code> to skip the int8 variant.

`predictor.py` — shared inference (~118 lines)

Symbol	Role
<code>softmax(z)</code>	Numerically stable softmax.
<code>SignPredictor</code>	Reads <code>model_meta.json</code> , rebuilds the EXACT <code>FeatureConfig</code> , runs ONNX (default) or torch. <code>features()</code> , <code>predict_features()</code> , <code>predict()</code> -> {name, conf, probs, topk}.
<code>video_to_clip(video_path, proc_width)</code>	Reuse <code>extract_dataset.extract_clip</code> + <code>build_all_vector</code> -> the same (T,1692) raw vector.

Used by **both** inference CLIs so the serving path is identical.

`infer_video.py` — STAGE 4: file inference (~228 lines)

Symbol	Role
<code>draw_frame(...)</code>	Draw saved landmarks on an original frame (matches verify-overlay look; reuses <code>landmark_detector</code> draw fns).
<code>_banner(...)</code>	Top (whole-clip pred) + bottom (current window) subtitle bars.
<code>timeline(raw, fps, ...)</code>	Sliding-window per-segment predictions.
<code>render_overlay(...)</code>	Write the prediction overlay <code>.mp4</code> .
<code>resolve(args)</code>	<code><class>/<stem></code> or an arbitrary <code>--path</code> -> video path + true label.
<code>run_one / main / parse_args</code>	Single clip, <code>--scan-dataset</code> batch test; <code>--backend onnx/torch --int8 --no-overlay --window --stride --proc-width</code> .

`infer_live.py` — STAGE 5: live translation (~236 lines)

Symbol	Role
<code>_banner(frame, lines)</code>	Live subtitle overlay.
<code>process_chunk(tmp_path, ...)</code>	Run extractor+model on a temp clip, gate on confidence + hands-seen, then delete the temp.
<code>run_webcam(args, predictor)</code>	Record->process->delete loop with live preview (Q to quit).
<code>run_source(args, predictor)</code>	Same loop simulated from a video file (no camera needed).
<code>_save_transcript(...)</code>	Write <code>live_transcript.json</code> ; clean up any leftover temp chunks.
<code>main / parse_args</code>	<code>--seconds --source --device-index --proc-width --backend --int8 --conf --no-display --keep-temp</code> .

05 — Data Formats

Input: the raw landmark clip

Read-only from the extraction project:

	Contents		
692)	per-frame `pose	face	left
	one row per clip; <code>verdict == FAIL</code> rows are skipped		
	<code>{ "good": 0, "hello": 1, ... }</code> class -> label		

The `(T, 1692)` layout (from `config.BLOCK`): pose `0-131` (33x4), face `132-1565` (478x3), left hand `1566-1628` (21x3), right hand `1629-1691` (21x3).

The model input: the `(L, D)` feature vector

`features.to_features()` turns each clip into a fixed `(L, D)` tensor. With the default `FeatureConfig` (`seq_len=64`, pose+hands, no face, velocity + masks):

Channel group	Width	Source
coordinates	165	55 points x 3 (13 pose joints + 21 left + 21 right)
pose visibility	13	one per curated pose joint
velocity	165	first-difference of the coordinates
presence masks	3	pose / left_hand / right_hand
Total D	346	

`L = 64` (fixed temporal length). So every clip -> `(64, 346)`, regardless of its original frame count. `D` changes if you toggle modalities (`--use-face`, etc.) — `features.feature_dim(cfg)` computes it for any config; the actual value used is recorded in `model_meta.json` (`in_dim`).

Coordinate space (after the transform)

- Centered on the clip's **mid-shoulder** and scaled by **shoulder width** ->

translation + scale invariant. (Not unit-variance; the model's input `LayerNorm` handles the rest.)

- Absent points sit exactly at the **origin** (re-zeroed after normalisation).
- `x` is negated under horizontal-flip augmentation, with L/R points swapped.

Checkpoints

File	Contents
<code>checkpoints/fold{k}.pt</code>	Per-fold best weights + <code>in_dim</code> , <code>classes</code> , <code>config</code> , <code>val_acc</code> .
<code>checkpoints/model_final.pt</code>	Deployment weights (<code>state_dict</code>) + full self-describing meta.
<code>checkpoints/model_meta.json</code>	The meta without weights (for inference).

`model_meta.json` (self-describing)

Written by `finetune.py` and copied beside the ONNX by `export_optimize.py`. Inference rebuilds the entire feature pipeline from this file alone.

```
{
  "kind": "final", "framework": "pytorch",
  "classes": ["good","hello", ... ],          // 12 classes
  "n_classes": 12,
  "in_dim": 346, "seq_len": 64, "raw_dim": 1692,
  "config": { "feature": {...}, "model": {...}, "train": {...} },
  "created": "2026-06-25 16:45:04",
  "fit_train_acc": 1.0, "weights": "ema",      // EMA vs raw chosen at fit time
  "cv_mean_acc": 0.9105, "cv_oof_accuracy": 0.9104,
  "onnx": "sign_model.onnx", "onnx_int8": "sign_model.int8.onnx",
  "onnx_vs_torch_max_diff": 1.67e-06
}
```

Exported models

File	Notes
<code>exported/sign_model.onnx</code>	fp32 ONNX, opset 17, dynamic batch, fixed L.
<code>exported/sign_model.int8.onnx</code>	Dynamic int8 quantized (smaller / faster on CPU).
<code>exported/model_meta.json</code>	Same meta + <code>onnx</code> / <code>onnx_int8</code> filenames + parity diff.

Metrics

File	Contents
<code>metrics/cv_report.json</code>	OOF accuracy + macro-F1, per-fold accs, per-class P/R/F1, confusion matrix, config.
<code>metrics/confusion_matrix.png</code>	Row-normalised OOF confusion matrix.
<code>metrics/cv_folds.png</code>	Per-fold accuracy bars.
<code>metrics/history.png</code>	Loss / val-acc curves per fold.

`cv_report.json` shape:

```
{
  "oof_accuracy": 0.9104, "oof_macro_f1": 0.9091,
  "cv_mean_acc": 0.9105, "cv_std_acc": 0.0291,
  "fold_val_acc": [0.9062, 0.8929, 0.9615, 0.875, 0.9167],
  "per_class": { "good": {"precision":0.9,"recall":0.818,"f1":0.857,"support":11}, ... },
  "confusion_matrix": [...], ...],
  "classes": [...], "in_dim": 346, "n_clips": 130, "config": {...}
}
```

Predictions

File	Contents
<code>predictions/<stem>_pred.mp4</code>	Overlay video: landmarks + predicted-sign subtitle.
<code>predictions/<stem>_pred.json</code>	<code>{prediction, conf, topk, timeline, correct}</code> .
<code>predictions/live_transcript.json</code>	<code>{transcript: [{t, sign, conf}, ...]}</code> from a live session.

Transient

- `live_tmp/chunk_*.mp4` — recorded webcam chunks, **deleted immediately** after each chunk is processed (a safety net also clears leftovers on exit).

- `cache/` — feature cache scratch space.

All of `training/artifacts/` is gitignored and regenerable by re-running the stages.

06 — Developer Guide

Setup

```
pip install torch onnxruntime numpy matplotlib mediapipe opencv-python
```

Python 3.11 recommended. The extraction project must already have produced `processed_dataset/` (run `python extract_dataset.py` first if not). Inference also imports `extract_dataset.py` and `landmark_detector.py` from the project root, so keep `training/` next to them.

Paths are resolved relative to the project root, so the commands below work from anywhere; the examples assume you run from the project root.

End-to-end run

```
python training/train.py          # 1. 5-fold CV -> metrics/cv_report.json
python training/finetune.py       # 2. fit on 100% -> checkpoints/model_final.pt
python training/export_optimize.py # 3. ONNX + int8 -> exported/sign_model.onnx
python training/infer_video.py --video good/good_03 # 4. file test + overlay
python training/infer_live.py     # 5. live webcam
```

Each stage prints the suggested next command when it finishes.

Everyday commands

```
# Train variants
python training/train.py --arch transformer --epochs 150
python training/train.py --use-face          # include the 1434-dim face block
python training/train.py --no-aug           # ablate augmentation
python training/train.py --seq-len 48 --hidden 96

# Deployment fit
python training/finetune.py --epochs 90
python training/finetune.py --init-from fold3 # warm-start from a fold checkpoint

# Export
python training/export_optimize.py --no-quant # skip the int8 variant

# File inference
python training/infer_video.py --video hello/hello --backend torch --no-overlay
python training/infer_video.py --path "C:/clips/test.mp4"
python training/infer_video.py --scan-dataset --limit 3 # batch sanity check

# Live (or simulate from a file without a camera)
python training/infer_live.py --seconds 2.0 --proc-width 480 --int8
python training/infer_live.py --source good.mp4 --no-display

# Module self-tests (quick smoke checks)
python training/features.py # feature spec + a sample clip + flip involution
python training/data.py    # dataset load + per-class counts
python training/model.py   # param count + forward shape
python training/predictor.py # load model + predict one sample
```

Common tasks

Change a hyperparameter

Edit the dataclass defaults in `config.py` (one source of truth), or pass a CLI flag for the common ones. The active config is recorded in `model_meta.json`, so exported models stay self-describing.

Add / remove a modality

Toggle `use_pose` / `use_hands` / `use_face` (or `--use-face`). The feature dimension `D`, flip permutations, and mask channels all adjust automatically — retrain and re-export. Inference picks up the new `D` from `model_meta.json`.

Add a new class

1. Add clips and re-run the extraction pipeline so `processed_dataset/` includes the new class (`classes.json`, `manifest.csv`, `landmarks_all/`).
2. Re-run `train.py` -> `finetune.py` -> `export_optimize.py`. The class list is read from `classes.json`, so no code change is needed.

Switch inference backend

`--backend onnx` (default, fast CPU) or `--backend torch`; add `--int8` for the quantized ONNX. If export hasn't run, `SignPredictor` falls back to the torch checkpoint automatically.

Debugging guide

Symptom	Likely cause / action
No clips found under <code>landmarks_all</code>	Extraction hasn't run, or <code>manifest.csv</code> is missing. Run <code>extract_dataset.py</code> .
<code>model_final.pt</code> not found (export)	Run <code>finetune.py</code> first.
ONNX export MISMATCH	A non-exportable op crept in; the legacy exporter handles GRU/Transformer — check custom layers.
int8 "quantization skipped"	<code>onnxruntime.quantization</code> missing/incompatible; use the fp32 ONNX or <code>--no-quant</code> .
Live always says "..."	Confidence below <code>--conf</code> , or hands not seen (>15% needed). Improve lighting/framing or lower <code>--conf</code> .
Low CV accuracy on one class	Inspect <code>confusion_matrix.png</code> — confusable signs share hand shape; collect more clips or enable face.
Slow live loop	Lower <code>--proc-width</code> (e.g. 320) and/or use <code>--int8</code> ; shorten <code>--seconds</code> .
<code>UnicodeEncodeError</code> on Windows	Handled — <code>utils._enable_utf8_console()</code> runs on import.

Inspect a model meta / features by hand

```
import sys; sys.path.insert(0, "training")
import numpy as np, features as F
from config import CONFIG, LANDMARKS_ALL
spec = F.build_spec(CONFIG.feature)
raw = np.load(next(LANDMARKS_ALL.glob("*/*.npy")))
print(raw.shape, "->", F.to_features(raw, CONFIG.feature, spec).shape) # (T,1692) -> (64,346)
```

Performance notes

- **Geometry is cached:** `load_geometric()` runs the expensive `geometric()` once per clip; each epoch only re-augments + assembles. CV over the whole dataset takes ~1–2 min on CPU.
- **CPU is the target:** there's no GPU here, so the int8 ONNX + ONNX Runtime is the fast path for live use (latency is reported by `export_optimize.py`).
- `--proc-width` is the main live speed knob; normalised coordinates are resolution-independent, so downscaling input barely affects accuracy.

Conventions for contributors

- Keep **all** feature logic in `features.py` and **all** model logic in

`model.py`; both training and inference must import them, never re-implement.

- Never duplicate the video->landmark path — always go through

`predictor.video_to_clip` (which reuses the verified extractor).

- Never hand-edit anything under `training/artifacts/` — regenerate it.
- Keep `model_meta.json` the single contract between training and inference.

07 — Results & Future Work

Measured results

From the latest 5-fold cross-validation ([metrics/cv_report.json](#), 134 clips, default BiGRU config, $D = 346$, $L = 64$):

- **Out-of-fold accuracy: 0.910, macro-F1: 0.909.**
- **Per-fold val accuracy: 0.906, 0.893, 0.962, 0.875, 0.917**

(mean 0.911 +/- 0.029).

- The deployment model ([finetune.py](#), EMA weights) fits the training clips at 1.00 and inherits the CV numbers in its meta.
- ONNX matches PyTorch to $\sim 1.7e-6$ max logit diff; int8 keeps top-1 agreement.

Per-class (out-of-fold)

Class	P	R	F1	n
good	0.90	0.82	0.86	11
hello	0.91	0.91	0.91	11
help	0.75	0.90	0.82	10
hi	0.85	1.00	0.92	11
how_are_you	0.92	0.92	0.92	13
me	0.82	0.90	0.86	10
nice_to_meet_you	1.00	0.91	0.95	11
no	1.00	0.92	0.96	13
sorry	0.89	0.80	0.84	10
thank_you	1.00	1.00	1.00	10
yes	1.00	0.92	0.96	12
you	0.92	0.92	0.92	12

The weakest classes ([help](#), [sorry](#), [good](#)) are the ones whose hand shape / motion overlaps with others — see [confusion_matrix.png](#). These are the first candidates for more data or richer features.

Numbers reflect the run that produced the committed artifacts. Re-running the stages regenerates them; expect small variation with seed/config changes.

Where to go next

Data

- **More clips per class.** ~11/class is the main limiter; even a handful more per class measurably stabilises the weak classes.
- **More signers / lighting / backgrounds.** Normalisation keeps features comparable across sources, so added variety mostly helps.
- **Balanced two-handed coverage.** Current data skews right-handed; flip-augmentation already mitigates this, but balanced collection helps more.

Features

- **Try the face block** (`--use-face`) for signs where expression matters — it is off by default to keep the model light (face is 1434/1692 raw dims).
- **Richer motion**: acceleration (second difference), or per-hand joint-angle features, on top of the current velocity channel.
- **Subset the face** via `face_idx` (e.g. mouth/brow only) to add expression cheaply.

Model

- **Transformer encoder** (`--arch transformer`) once the dataset is larger — it has more capacity than the BiGRU but needs more data to pay off.
- **Temperature calibration** so the live confidence threshold is more meaningful across classes.

Inference / product

- **Continuous segmentation**. The live loop classifies fixed chunks; a proper sequence segmenter (or CTC-style decoding) would handle sign boundaries instead of a sliding window.
- **Streaming model**. A causal/streaming variant would cut the chunk latency of the record->process->delete loop.
- **On-device export**. The int8 ONNX is already CPU-friendly; ORT-Mobile / WebGPU would extend this to phones / browsers.

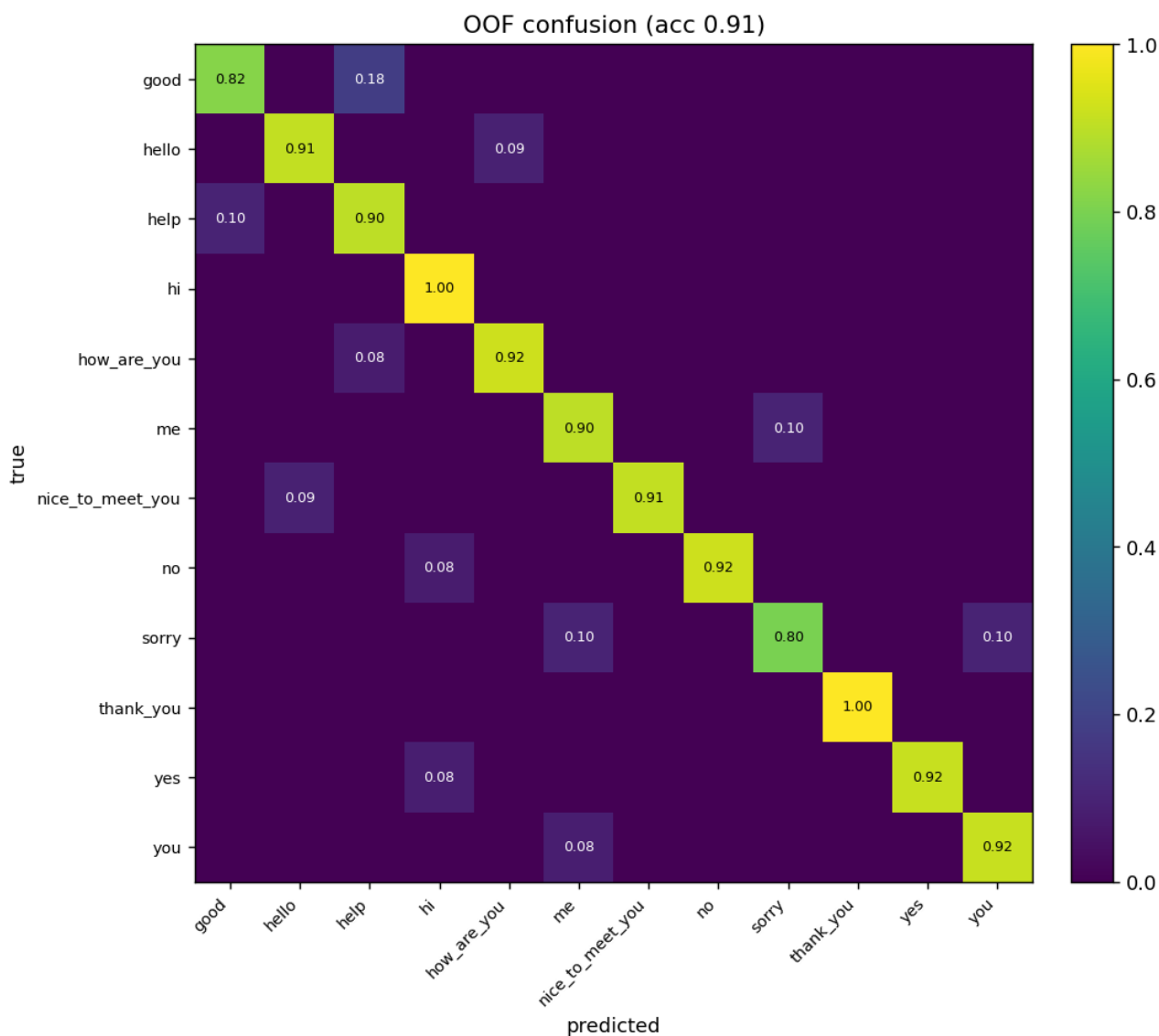
Evaluation

- **Confusion-driven collection loop**: use `confusion_matrix.png` to target exactly the pairs the model confuses.
- **Held-out signer test** once multiple signers exist — the honest measure of generalisation beyond k-fold over a single signer.

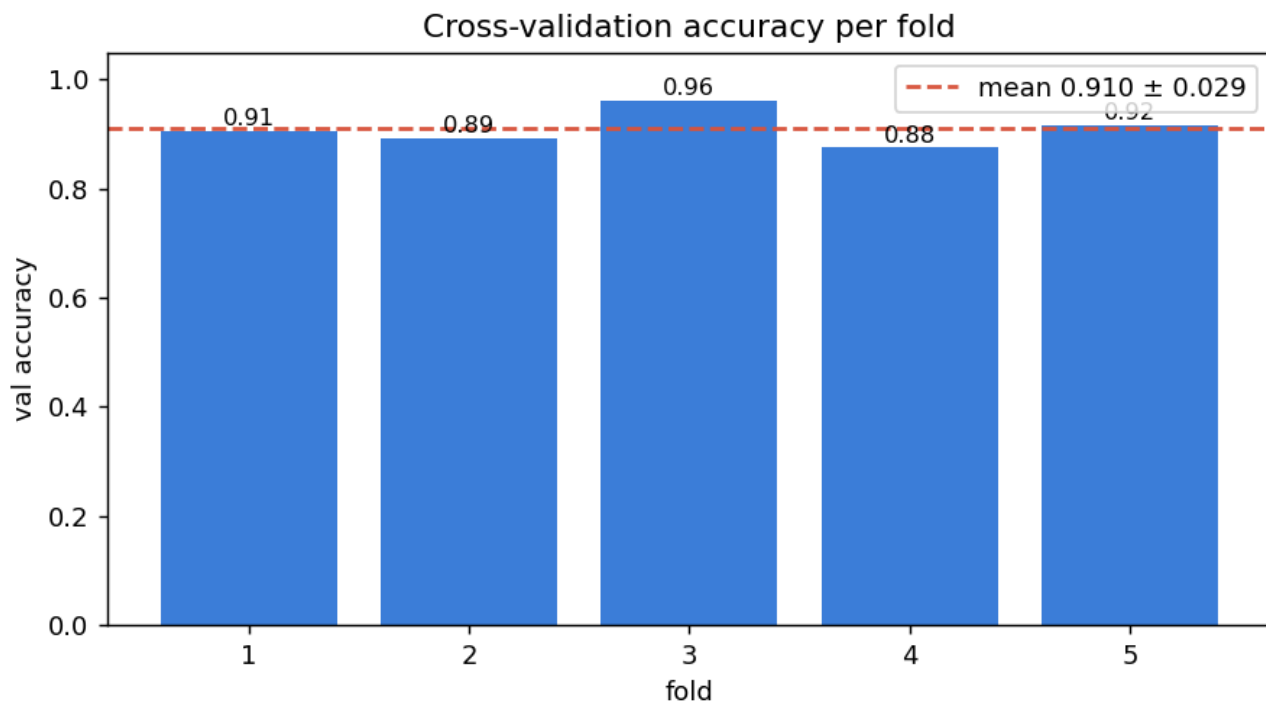
Appendix — Training Metrics

Auto-generated charts from `train.py` (cross-validation) under `training/artifacts/metrics/`.

Confusion Matrix



Cv Folds



History

Training history

